

Backend Mastery Spring Boot vs Node.js

Basic to Advanced | Concepts, Syntax, Examples
Detailed Comparison and Decision Guide

Contents

How to Use This Guide	5
The 3-volume series	5
How each chapter is organised	5
Versions this guide targets	5
Suggested study plan	5
 1. The Backend Landscape	 6
1.1 What is a backend?	6
1.2 Java + Spring Boot in one paragraph	6
1.3 Node.js in one paragraph	6
1.4 Runtime models compared	6
1.5 Thread-per-request vs event loop (the key idea)	7
1.6 Ecosystem map	7
 2. Spring Boot - Basics	 8
2.1 Creating a project	8
2.2 The entry point	8
2.3 Your first REST controller	8
2.4 Configuration: application.yml and profiles	9
2.5 Dependency Injection (DI) and IoC	10
2.6 How auto-configuration works	11
2.7 Useful development tools	11
 3. Node.js - Basics	 12
3.1 Install and run	12
3.2 Modules: CommonJS vs ES Modules	12
3.3 The event loop (must-know concept)	12
3.4 Async patterns: callbacks, promises, async/await	13
3.5 Core built-in modules	13
3.6 A raw HTTP server (no framework)	14
3.7 Your first Express API (TypeScript)	14
3.8 Environment configuration	15
3.9 Choosing a Node web framework	15
 4. Side-by-Side: Basics Compared	 16
4.1 Hello endpoint	16
4.2 Concept mapping table	16
4.3 Language syntax cheat sheet (Java vs TypeScript)	17
4.4 Same task, three ways: validation	17
4.5 Quick decision guide (basics level)	18

5. Spring Boot - Intermediate	19
5.1 Layered architecture	19
5.2 JPA entity	19
5.3 Relationships	19
5.4 Spring Data JPA repositories	20
5.5 Service layer with transactions	21
5.6 DTOs and mapping	21
5.7 Controller with validation, pagination and proper status codes	21
5.8 Global exception handling (RFC 9457 Problem Details)	22
5.9 Database migrations	22
6. Node.js - Intermediate	24
6.1 Recommended project structure	24
6.2 Middleware: the heart of Express	24
6.3 Router, controller, service	25
6.4 Centralised error handling	25
6.5 Database access: ORM comparison	25
6.6 Spring Data JPA vs Prisma (mapping)	26
6.7 NestJS: Spring-style structure for Node	27
6.7.1 Pagination patterns (both stacks)	27
7. Security and Authentication	28
7.1 Core concepts	28
7.2 JWT structure	28
7.3 Spring Security (6.x / 7.x) with JWT resource server	28
7.4 Node.js: JWT auth with Express and jose	29
7.5 Security comparison	30
7.6 OWASP-style checklist (applies to both)	30
8. Testing	31
8.1 The test pyramid	31
8.2 Spring Boot testing	31
8.3 Node.js testing with Vitest and Supertest	32
8.4 Testing comparison	33
9. Spring Boot - Advanced	34
9.1 Caching	34
9.2 Async, scheduling and events	34
9.3 AOP (Aspect-Oriented Programming)	35
9.4 Virtual threads (Java 21+) - the game changer	35
9.5 Reactive stack: Spring WebFlux	35
9.6 HTTP clients and resilience	36

9.7 Messaging with Kafka	36
9.8 Observability with Actuator, Micrometer and OpenTelemetry	37
9.9 Performance tuning checklist	37
9.10 Native images and containers	37
9.11 Microservices with Spring	38
10. Node.js - Advanced	39
10.1 Streams and backpressure	39
10.2 Worker threads for CPU-bound work	39
10.3 Scaling across CPU cores	39
10.4 Fastify: performance-oriented alternative	40
10.5 Background jobs and queues (BullMQ + Redis)	40
10.6 Real-time: WebSockets and Server-Sent Events	40
10.7 Graceful shutdown and health checks	41
10.8 Performance and memory	41
10.9 Observability	41
10.10 Production Dockerfile	42
10.11 Advanced Node patterns worth knowing	42
11. Deep Comparison: Spring Boot vs Node.js	43
11.1 Master comparison table	43
11.2 Performance in context (honest view)	43
11.3 Cost of running (typical trend, not exact numbers)	44
11.4 Code volume: the same feature	44
11.5 When to choose which - decision matrix	44
11.6 Hybrid architecture (common in real companies)	45
12. Full Example: Task API in Both Stacks	46
12.1 API contract	46
12.2 Spring Boot version (complete)	46
12.3 Node.js version (Express + Prisma + Zod, complete)	47
12.4 Observations from the comparison	48
13. Interview Q&A, Cheat Sheet and Roadmap	49
13.1 Spring Boot interview questions	49
13.2 Node.js interview questions	49
13.3 Common mistakes	50
13.4 Command cheat sheet	50
13.5 Learning roadmap (basic to advanced)	51

How to Use This Guide

CATEGORY

Volume 1 of 3 - Backend Engineering. Covers Java Spring Boot and Node.js (Express, Fastify, NestJS) from first principles to production-grade architecture, with side-by-side comparisons. Level: Beginner to Advanced.

The 3-volume series

Volume	Topics	Category
Vol 1 - Backend (this file)	Spring Boot, Node.js, Express, NestJS	Server-side / API / Business logic
Vol 2 - Language and UI	TypeScript, React (basic to advanced)	Language + Client-side UI
Vol 3 - Full-stack and Styling	Next.js, Tailwind CSS, full-stack comparison, decision guides	Framework + Styling + Architecture

How each chapter is organised

- **Category box** at the top of every chapter: what the topic is, where it is used, and its difficulty level.
- **Concept** explained in plain words first, then **syntax**, then a **worked example**.
- **Comparison tables** between Spring Boot and Node.js appear after each major topic, so you learn both at once.
- **Warning / Pro tip / Note** boxes highlight common mistakes, optimisation tips, and version differences.

Versions this guide targets

Technology	Version used	Notes
Java	21 LTS (17 minimum for Boot 3/4)	Records, pattern matching, virtual threads
Spring Boot	3.5.x and 4.0.x	4.0 uses Spring Framework 7, Hibernate 7, Jackson 3
Node.js	22 / 24 LTS	Built-in test runner, fetch, ESM by default in new code
TypeScript	5.x	Type stripping supported natively in recent Node versions
Express	4.x / 5.x	Express 5 handles rejected promises in handlers
NestJS	10 / 11	Opinionated framework similar to Spring in style

NOTE

Framework versions move fast. The concepts (DI, layers, event loop, transactions) stay stable; only annotations, package names and defaults change. When in doubt, check the official docs for your exact version.

Suggested study plan

1. **Week 1 - Basics:** Chapters 2 to 4. Build the "Hello API" in both stacks.
2. **Week 2 - Intermediate:** Chapters 5 and 6. Build a CRUD "Task API" with a database.
3. **Week 3 - Security and testing:** Chapters 7 and 8.
4. **Week 4 - Advanced and comparison:** Chapters 9 to 12. Deploy with Docker.

1. The Backend Landscape

CATEGORY

Foundations. What a backend does, how the JVM and Node.js runtimes differ, and how to think about "which one should I use". Level: Beginner.

1.1 What is a backend?

A backend is the program that runs on a server and does the work the user never sees: it receives HTTP requests, validates them, applies business rules, talks to databases and other services, and returns a response (usually JSON).

TEXT

```

Browser / Mobile App
  | HTTP request (GET /api/tasks/5)
  v
+-----+
| Controller / Route handler | <- parses request, returns response
| Service (business logic)   | <- rules, calculations, transactions
| Repository / DAO          | <- talks to the database
+-----+
  | SQL / queries
  v
Database (PostgreSQL, MySQL, MongoDB)
  
```

1.2 Java + Spring Boot in one paragraph

Java is a statically typed, compiled language that runs on the **JVM** (Java Virtual Machine). **Spring** is a huge framework built around **Inversion of Control (IoC)** and **Dependency Injection (DI)**. **Spring Boot** sits on top of Spring and removes boilerplate: it gives you *starters* (ready dependency bundles), *auto-configuration* (sensible defaults), an *embedded server* (Tomcat) and *Actuator* (production monitoring).

1.3 Node.js in one paragraph

Node.js is a JavaScript runtime built on Google's **V8 engine**. It executes JavaScript outside the browser and uses a **single-threaded event loop** with non-blocking I/O, so one process can handle thousands of concurrent connections. Frameworks such as **Express**, **Fastify** and **NestJS** add routing, middleware and structure.

1.4 Runtime models compared

Aspect	Spring Boot (JVM)	Node.js (V8)
Language	Java / Kotlin (static, compiled)	JavaScript / TypeScript (dynamic; TS is compiled away)
Concurrency model	Thread-per-request (platform threads) or virtual threads (Java 21)	Single-threaded event loop + libuv thread pool
Best at	CPU-heavy work, complex domains, large teams, long-lived services	I/O-heavy APIs, real-time, gateways, BFFs, rapid prototyping
Startup time	Slower (1-5 s typical; ms with GraalVM native)	Fast (tens to hundreds of ms)
Memory footprint	Higher (JVM heap; often 200-500 MB baseline)	Lower (often 50-150 MB baseline)
Type safety	Compile-time, very strong	Only with TypeScript (compile-time)
Package manager	Maven / Gradle (Maven Central)	npm / pnpm / yarn (npm registry)

Aspect	Spring Boot (JVM)	Node.js (V8)
Build output	JAR / native image	Source files or bundled JS
Structure	Convention-heavy (layers, annotations)	Flexible (you choose); NestJS adds structure
Learning curve	Steeper (Java + Spring concepts)	Gentler start, harder to keep large codebases tidy

1.5 Thread-per-request vs event loop (the key idea)

TEXT

```

Spring Boot (classic):
request1 -> Thread-1 (waits on DB)
request2 -> Thread-2 (waits on DB)
request3 -> Thread-3 (waits on DB)
(each thread costs ~1 MB stack)

Node.js:
request1 \
request2  >-> ONE thread + event loop
request3 /   (DB wait does not block it)
(callbacks/promises resume when I/O ends)

```

- **Thread-per-request:** simple mental model, real parallelism on many CPU cores, but threads are expensive when you have 10,000 waiting connections. *Virtual threads (Java 21+)* fix this: millions of cheap threads, same blocking-style code.
- **Event loop:** extremely efficient for waiting on I/O, but a single **CPU-heavy** function (big JSON parse, image resize, crypto loop) **blocks every other request**. Fix with worker threads or a separate service.

WARNING

In Node.js never run long synchronous loops in a request handler. One slow handler stalls the entire process. In Spring Boot a slow request only occupies one thread.

1.6 Ecosystem map

Need	Spring Boot	Node.js
Web / REST	Spring Web MVC, Spring WebFlux	Express, Fastify, Koa, Hono, NestJS
Database access	Spring Data JPA (Hibernate), JDBC Template, jOOQ, MyBatis	Prisma, TypeORM, Drizzle, Sequelize, Knex, Mongoose
Validation	Jakarta Bean Validation (@Valid)	Zod, Joi, class-validator, Yup
Security	Spring Security, OAuth2 Resource Server	Passport, jose, bcrypt, Helmet, NestJS Guards
Testing	JUnit 5, Mockito, MockMvc, Testcontainers	Vitest, Jest, node:test, Supertest
Messaging	Spring Kafka, Spring AMQP	KafkaJS, amqplib, BullMQ
Docs	springdoc-openapi (Swagger UI)	swagger-jsdoc, @nestjs/swagger, zod-to-openapi
Config	application.yml, profiles, @ConfigurationProperties	dotenv, process.env , config libs, NestJS ConfigModule
Monitoring	Actuator + Micrometer	prom-client, OpenTelemetry, pino

2. Spring Boot - Basics

CATEGORY

Framework: Java backend. Project setup, core annotations, first REST endpoint, configuration and profiles. Used for enterprise APIs, banking, e-commerce, microservices. Level: Beginner.

2.1 Creating a project

Go to **start.spring.io** (Spring Initializr) or use your IDE. Choose: Maven or Gradle, Java 21, Spring Boot 3.5.x / 4.0.x, and dependencies: **Spring Web, Spring Data JPA, Validation, PostgreSQL Driver** (or H2 for learning), **Spring Boot Actuator, Lombok** (optional).

TEXT

```
taskapi/  
  pom.xml (or build.gradle)  
  src/main/java/com/example/taskapi/  
    TaskApiApplication.java <- entry point  
    controller/ service/ repository/ model/ dto/ exception/ config/  
  src/main/resources/  
    application.yml  
  src/test/java/...
```

2.2 The entry point

JAVA

```
package com.example.taskapi;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration + @ComponentScan  
public class TaskApiApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(TaskApiApplication.class, args);  
    }  
}
```

@SpringBootApplication does three things: (1) marks the class as a source of bean definitions, (2) turns on **auto-configuration** (Boot inspects your classpath and configures things), (3) **component-scans** the current package and sub-packages for **@Component**, **@Service**, **@Repository**, **@RestController**.

2.3 Your first REST controller

JAVA

```

@RestController // @Controller + @ResponseBody: return values become JSON
@RequestMapping("/api/hello")
public class HelloController {

    @GetMapping // GET /api/hello
    public String hello() {
        return "Hello, Spring Boot!";
    }

    @GetMapping("/{name}") // GET /api/hello/Asha
    public Map<String, String> greet(@PathVariable String name,
                                    @RequestParam(defaultValue = "en") String lang) {
        return Map.of("message", "Hello " + name, "lang", lang);
    }

    @PostMapping // POST /api/hello (JSON body)
    @ResponseStatus(HttpStatus.CREATED)
    public GreetRequest create(@RequestBody GreetRequest body) {
        return body;
    }

    public record GreetRequest(String name, int age) {} // Java record = immutable DTO
}

```

Run with `./mvnw spring-boot:run` (or `./gradlew bootRun`). The app starts on **http://localhost:8080**.

Request mapping annotations

Annotation	Purpose	Example
<code>@GetMapping</code>	HTTP GET	<code>@GetMapping("/tasks")</code>
<code>@PostMapping</code>	HTTP POST	<code>@PostMapping("/tasks")</code>
<code>@PutMapping</code>	Full update	<code>@PutMapping("/tasks/{id}")</code>
<code>@PatchMapping</code>	Partial update	<code>@PatchMapping("/tasks/{id}")</code>
<code>@DeleteMapping</code>	Delete	<code>@DeleteMapping("/tasks/{id}")</code>
<code>@PathVariable</code>	URL segment to parameter	<code>/tasks/{id}</code>
<code>@RequestParam</code>	Query string parameter	<code>?page=2</code>
<code>@RequestBody</code>	JSON body to object	<code>@RequestBody TaskDto dto</code>
<code>@RequestHeader</code>	Read a header	<code>@RequestHeader("X-Trace-Id")</code>
<code>ResponseEntity<T></code>	Full control of status + headers + body	<code>ResponseEntity.ok(body)</code>

2.4 Configuration: application.yml and profiles

YAML

```
server:
  port: 8080

spring:
  application:
    name: taskapi
  datasource:
    url: jdbc:postgresql://localhost:5432/tasks
    username: ${DB_USER:postgres}      # env var with default
    password: ${DB_PASS:secret}
  jpa:
    hibernate:
      ddl-auto: validate                # none | validate | update | create | create-drop
      open-in-view: false               # recommended: avoid lazy loading in the web layer
    properties:
      hibernate.jdbc.batch_size: 50
  management:
    endpoints:
      web:
        exposure:
          include: health,info,metrics,prometheus
```

Profiles let you switch settings per environment: create `application-dev.yml`, `application-prod.yml` and activate with `SPRING_PROFILES_ACTIVE=prod` (or `--spring.profiles.active=prod`).

Type-safe configuration

JAVA

```
@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(String secret, Duration expiry) {}

// application.yml: app.jwt.secret: abc | app.jwt.expiry: 15m
// Enable with @EnableConfigurationProperties(JwtProperties.class) or @ConfigurationPropertiesScan
```

2.5 Dependency Injection (DI) and IoC

Concept: instead of a class creating its own dependencies (`new TaskRepository()`), Spring creates objects (**beans**), stores them in the **ApplicationContext**, and *injects* them where needed. This makes code loosely coupled and easy to test (swap in a mock).

JAVA

```
@Service // stereotype: registers a bean
public class TaskService {

    private final TaskRepository repo; // final = immutable, required

    // Constructor injection (RECOMMENDED). Single constructor -> no @Autowired needed.
    public TaskService(TaskRepository repo) {
        this.repo = repo;
    }
}
```

Injection style	Recommended?	Why
Constructor	Yes (best)	Immutable, mandatory deps, easy unit tests, fails fast
Setter	Rarely	Optional dependencies only
Field (@Autowired on field)	No	Hard to test, hides dependencies, cannot be final

Stereotype	Layer	Extra behaviour
@Component	Generic bean	None
@Service	Business logic	Semantic only
@Repository	Data access	Translates persistence exceptions
@RestController	Web layer	@ResponseBody on every method

Stereotype	Layer	Extra behaviour
@Configuration + @Bean	Manual bean definitions	Create third-party beans

JAVA

```
@Configuration
public class AppConfig {
    @Bean
    public Clock clock() { return Clock.systemUTC(); }    // now injectable everywhere
}
```

Bean scopes

Scope	Meaning
singleton (default)	One instance per application context
prototype	New instance every time it is requested
request / session	One per HTTP request / session (web apps)

NOTE

Beans are **singletons** shared by many threads. Never keep mutable per-request state in fields of a `@Service`.

2.6 How auto-configuration works

Boot reads

`META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from every JAR on the classpath. Each auto-config class is guarded by conditions:

JAVA

```
@AutoConfiguration
@ConditionalOnClass(DataSource.class)    // only if JDBC is on classpath
@ConditionalOnMissingBean(DataSource.class)    // back off if YOU defined one
public class DataSourceAutoConfiguration { /* ... */ }
```

PRO TIP

Start the app with `--debug` (or `debug=true`) to print the **conditions report**: it lists which auto-configurations matched and why. This is the fastest way to solve "why is this bean missing / present?".

2.7 Useful development tools

- **spring-boot-devtools**: automatic restart on class change.
- **Actuator**: `/actuator/health`, `/actuator/metrics`, `/actuator/env`.
- **springdoc-openapi**: add `springdoc-openapi-starter-webmvc-ui` and open `/swagger-ui.html`.
- **Docker Compose support (Boot 3.1+)**: put a `compose.yaml` next to your project; Boot starts the containers and wires the connection automatically.

3. Node.js - Basics

CATEGORY

Runtime: JavaScript on the server. Modules, npm, the event loop, async patterns, and a first Express API. Used for REST APIs, real-time apps (chat, live dashboards), API gateways, tooling and serverless functions. Level: Beginner.

3.1 Install and run

Install the **LTS** version from nodejs.org (or use **nvm** / **fnm**). Then:

BASH

```
node -v           # check version
npm init -y       # creates package.json
npm install express # add a dependency
npm install -D typescript tsx @types/node @types/express # dev dependencies
node index.js     # run a file
npx tsx watch src/index.ts # run TypeScript directly with auto-reload
```

package.json essentials

JSON

```
{
  "name": "taskapi",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc -p .",
    "start": "node dist/index.js",
    "test": "vitest run"
  },
  "dependencies": { "express": "^5.0.0", "zod": "^3.23.0" },
  "devDependencies": { "typescript": "^5.6.0", "tsx": "^4.19.0", "vitest": "^2.0.0" }
}
```

"type": "module" turns on ES Modules (**import/export**). Without it Node treats **.js** files as CommonJS (**require**).

3.2 Modules: CommonJS vs ES Modules

Feature	CommonJS (older)	ES Modules (modern)
Import	<code>const fs = require("fs")</code>	<code>import fs from "node:fs"</code>
Export	<code>module.exports = fn</code>	<code>export default fn / export const x</code>
Loading	Synchronous	Asynchronous, static analysis
Top-level await	Not allowed	Allowed
File extension	<code>.js / .cjs</code>	<code>.js</code> (with type: module) / <code>.mjs</code>
<code>__dirname</code>	Available	Use <code>import.meta.dirname</code> (Node 20.11+)

3.3 The event loop (must-know concept)

Node runs your JavaScript on **one thread**. When you start an I/O operation (file read, DB query, HTTP call), Node hands it to the OS or a thread pool (libuv) and continues. When the operation finishes, its callback is queued and the loop runs it.

TEXT

```

+-----+
| Call stack | <- runs your sync code, one thing at a time
+-----+
|           |
+-----V-----+
| Microtask queue | <- Promise .then / await continuations, queueMicrotask
| process.nextTick queue | (run BEFORE the next macrotask)
+-----+
|           |
+-----V-----+
| Event loop phases: |
| timers -> pending -> |
| poll (I/O) -> check | <- setImmediate
| (setImmediate) -> close |
+-----+

```

JS

```

console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0); // macrotask (timers phase)
setImmediate(() => console.log("5 immediate")); // macrotask (check phase)
Promise.resolve().then(() => console.log("3 promise")); // microtask
process.nextTick(() => console.log("2 nextTick")); // runs first among async
// Typical output order: 1, 2, 3, 4, 5

```

3.4 Async patterns: callbacks, promises, async/await

JS

```

import { readFile } from "node:fs/promises";

// 1) Callback (old style - "callback hell")
fs.readFile("a.txt", "utf8", (err, data) => { if (err) return console.error(err); });

// 2) Promise
readFile("a.txt", "utf8").then(txt => console.log(txt)).catch(console.error);
// 3) async/await (preferred)
async function load() {
  try {
    const txt = await readFile("a.txt", "utf8");
    return txt.toUpperCase();
  } catch (err) {
    console.error("read failed", err);
    throw err;
  }
}

// Parallel work - do NOT await one by one if they are independent
const [users, orders] = await Promise.all([fetchUsers(), fetchOrders()]);
// Promise.allSettled -> never rejects; Promise.race -> first to finish; Promise.any -> first success

```

WARNING

`await` inside a `for` loop runs sequentially. For 100 independent requests use `Promise.all` (with a concurrency limit such as `p-limit` if the list is large).

3.5 Core built-in modules

Module	Use
<code>node:fs</code> / <code>node:fs/promises</code>	Files and directories
<code>node:path</code>	Safe path joining (<code>path.join</code>)
<code>node:http</code> / <code>node:https</code>	Low-level HTTP server and client
<code>node:events</code>	<code>EventEmitter</code> (publish/subscribe inside a process)
<code>node:stream</code>	Process large data piece by piece

Module	Use
<code>node:crypto</code>	Hashing, random bytes, UUID (<code>crypto.randomUUID()</code>)
<code>node:worker_threads</code>	Real parallel threads for CPU work
<code>node:child_process</code>	Spawn other programs
<code>node:test</code>	Built-in test runner (<code>node --test</code>)
<code>node:util</code>	<code>promisify</code> , <code>inspect</code> , <code>parseArgs</code>

Modern Node also has global `fetch`, `AbortController`, `structuredClone`, `URL`, `Blob` and `--watch` mode (`node --watch app.js`).

3.6 A raw HTTP server (no framework)

JS

```
import { createServer } from "node:http";

const server = createServer((req, res) => {
  if (req.method === "GET" && req.url === "/api/hello") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "Hello, Node!" }));
    return;
  }
  res.writeHead(404).end();
});

server.listen(3000, () => console.log("http://localhost:3000"));
```

Doing routing, body parsing and validation by hand quickly becomes painful, which is why frameworks exist.

3.7 Your first Express API (TypeScript)

TS

```
import express, { Request, Response, NextFunction } from "express";

const app = express();
app.use(express.json()); // parse JSON bodies

type Greet = { name: string; age: number };

app.get("/api/hello", (_req: Request, res: Response) => {
  res.json({ message: "Hello, Express!" });
});

app.get("/api/hello/:name", (req: Request, res: Response) => {
  const lang = (req.query.lang as string) ?? "en"; // ?lang=fr
  res.json({ message: `Hello ${req.params.name}`, lang });
});

app.post("/api/hello", (req: Request<{}>, {}, Greet, res: Response) => {
  res.status(201).json(req.body);
});

// Error-handling middleware: 4 parameters
app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
  console.error(err);
  res.status(500).json({ error: "Internal Server Error" });
});

app.listen(3000, () => console.log("Listening on http://localhost:3000"));
```

3.8 Environment configuration

TS

```
// .env (never commit secrets):  PORT=3000  DATABASE_URL=postgres://...
// Node 20.6+ can load it natively:  node --env-file=.env dist/index.js
import { z } from "zod";

const Env = z.object({
  PORT: z.coerce.number().default(3000),
  DATABASE_URL: z.string().url(),
  NODE_ENV: z.enum(["development", "test", "production"]).default("development"),
});
export const env = Env.parse(process.env);    // crashes at startup if config is wrong (good!)
```

3.9 Choosing a Node web framework

Frame work	Style	Strengths	Weaknesses
Express	Minimal, middleware-based	Huge ecosystem, everyone knows it	No structure, slower than Fastify, few built-ins
Fastify	Minimal, plugin-based	Very fast, JSON-schema validation, great TypeScript support	Smaller ecosystem than Express
Koa / Hono	Tiny, modern	Lightweight, Hono runs on edge runtimes	You assemble everything
NestJS	Opinionated (Angular / Spring style)	Modules, DI, decorators, structure for big teams	More boilerplate, learning curve

4. Side-by-Side: Basics Compared

CATEGORY

Comparison chapter. The same everyday tasks written in Spring Boot and Node.js so you can map one to the other instantly. Level: Beginner to Intermediate.

4.1 Hello endpoint

JAVA

```
// Spring Boot
@RestController
class HelloController {
    @GetMapping("/hello/{name}")
    Map<String, String> hello(@PathVariable String name) {
        return Map.of("message", "Hello " + name);
    }
}
```

TS

```
// Express + TypeScript
app.get("/hello/:name", (req, res) => {
    res.json({ message: `Hello ${req.params.name}` });
});
```

TS

```
// NestJS
@Controller("hello")
export class HelloController {
    @Get(":name")
    hello(@Param("name") name: string) {
        return { message: `Hello ${name}` };
    }
}
```

4.2 Concept mapping table

Concept	Spring Boot	Express	NestJS
Entry point	@SpringBootApplication main class	app.listen()	NestFactory.create(AppModule)
Route	@GetMapping("/x")	app.get("/x", fn)	@Get("/x")
Path param	@PathVariable Long id	req.params.id	@Param("id") id
Query param	@RequestParam int page	req.query.page	@Query("page") page
Body	@RequestBody Dto dto	req.body	@Body() dto
Header	@RequestHeader("X")	req.header("X")	@Headers("x")
Status	@ResponseStatus(CREATED)	res.status(201)	@HttpCode(201)
Interceptor / middleware	HandlerInterceptor, OncePerRequestFilter	app.use(fn)	Middleware, Interceptors
Dependency injection	Built in (IoC container)	Manual or library (awilix, tsyringe)	Built in (providers)
Validation	@Valid + @NotBlank	Zod / Joi in middleware	ValidationPipe + class-validator
Global error handling	@RestControllerAdvice	Error middleware (err, req, res, next)	Exception filters

Concept	Spring Boot	Express	NestJS
Auth guard	Spring Security filter chain	Auth middleware	@UseGuards(AuthGuard)
Config	application.yml, @Value, @ConfigurationProperties	process.env, dotenv	ConfigModule
Scheduling	@Scheduled(cron = "...")	node-cron, BullMQ	@Cron() from @nestjs/schedule
Logging	SLF4J + Logback	pino, winston	Built-in Logger / pino
Testing web layer	MockMvc, @WebMvcTest	Supertest	@nestjs/testing + Supertest
Build tool	Maven / Gradle	npm scripts, tsc, tsup, esbuild	Nest CLI

4.3 Language syntax cheat sheet (Java vs TypeScript)

Topic	Java 21	TypeScript
Variable	<code>String name = "A"; var x = 5;</code>	<code>const name: string = "A"; let x = 5;</code>
Constant	<code>final int MAX = 10;</code>	<code>const MAX = 10;</code>
Function	<code>int add(int a, int b) { return a + b; }</code>	<code>const add = (a: number, b: number): number => a + b;</code>
Class	<code>class User { private String name; }</code>	<code>class User { constructor(private name: string) {} }</code>
Interface	<code>interface Repo { void save(User u); }</code>	<code>interface Repo { save(u: User): void }</code>
Immutable data	<code>record User(String name, int age) {}</code>	<code>type User = Readonly<{ name: string; age: number }></code>
Enum	<code>enum Status { OPEN, DONE }</code>	<code>type Status = "OPEN" "DONE" (or enum)</code>
Generics	<code>List<String> xs = new ArrayList<>();</code>	<code>const xs: string[] = []; Array<string></code>
Null handling	<code>Optional<User></code>	<code>User undefined, ?. and ??</code>
Collections	<code>List, Set, Map</code>	<code>Array, Set, Map, object literal</code>
Stream / map	<code>list.stream().map(x -> x * 2).toList()</code>	<code>list.map(x => x * 2)</code>
String template	<code>"Hi %s".formatted(n) / text blocks</code>	<code>` Hi \${n} `</code>
Exceptions	<code>throw new IllegalStateException("x")</code>	<code>throw new Error("x")</code>
Async	<code>CompletableFuture<T>, virtual threads</code>	<code>Promise<T>, async / await</code>
Type check	<code>obj instanceof User u (pattern matching)</code>	<code>typeof x === "string", x instanceof User</code>
Equality	<code>.equals()</code> for objects	<code>=== (strict)</code>

4.4 Same task, three ways: validation

JAVA

```
// Spring Boot: annotations on the DTO
public record CreateTask(
    @NotBlank @Size(max = 100) String title,
    @Future LocalDate dueDate,
    @Min(1) @Max(5) int priority) {}

@PostMapping("/tasks")
Task create(@Valid @RequestBody CreateTask dto) { ... } // 400 automatically on failure
```

TS

```
// Express + Zod: schema is the single source of truth (types are inferred)
const CreateTask = z.object({
  title: z.string().min(1).max(100),
  dueDate: z.coerce.date().min(new Date()),
  priority: z.number().int().min(1).max(5),
});
type CreateTask = z.infer<typeof CreateTask>;
app.post("/tasks", (req, res) => {
  const parsed = CreateTask.safeParse(req.body);
  if (!parsed.success) return res.status(400).json(parsed.error.flatten());
  res.status(201).json(parsed.data);
});
```

TS

```
// NestJS: class-validator decorators (very similar to Java)
export class CreateTaskDto {
  @IsString() @IsNotEmpty() @MaxLength(100) title: string;
  @IsDateString() dueDate: string;
  @IsInt() @Min(1) @Max(5) priority: number;
}
```

4.5 Quick decision guide (basics level)

If you...	Lean towards
Already know Java / C# and want strict structure	Spring Boot
Know JavaScript / React and want one language across the stack	Node.js + TypeScript
Are building a quick prototype or MVP	Node.js (Express / Fastify)
Are building a large, long-lived enterprise system with many developers	Spring Boot (or NestJS)
Need real-time features (WebSockets, streaming) with minimal effort	Node.js
Need heavy computation, batch jobs, complex transactions	Spring Boot

5. Spring Boot - Intermediate

CATEGORY

Building real applications. Layered architecture, JPA/Hibernate, relationships, queries, pagination, DTO mapping, validation, exception handling and transactions. Level: Intermediate.

5.1 Layered architecture

TEXT

```
Controller  -> Service  -> Repository  -> Database
  (HTTP)      (rules)    (queries)
  |           |           |
  DTOs       Entities   Entities
```

- **Controller:** only HTTP concerns (parse, validate, map to DTO, return status). No business rules.
- **Service:** business rules and transactions (`@Transactional`).
- **Repository:** persistence only.
- **Entity vs DTO:** never expose JPA entities directly in APIs. Use DTOs (records) so database changes do not break clients, and to avoid lazy-loading and infinite JSON recursion.

5.2 JPA entity

JAVA

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)    // auto-increment
    private Long id;

    @Column(nullable = false, length = 100)
    private String title;

    @Enumerated(EnumType.STRING)                        // store "OPEN", not 0
    private Status status = Status.OPEN;

    private LocalDate dueDate;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "owner_id")
    private AppUser owner;

    @Version                                           // optimistic locking
    private Long version;

    @CreationTimestamp private Instant createdAt;
    protected Task() {}                               // JPA needs a no-arg constructor
    public Task(String title, AppUser owner) { this.title = title; this.owner = owner; }
    // getters / setters ...
    public enum Status { OPEN, IN_PROGRESS, DONE }
}
```

5.3 Relationships

Type	Annotation	Example	Notes
One-to-Many	<code>@OneToMany(mappedBy = "owner")</code>	User has many Tasks	Owning side is the <code>@ManyToOne</code>
Many-to-One	<code>@ManyToOne(fetch = LAZY)</code>	Task belongs to User	Always set LAZY explicitly
One-to-One	<code>@OneToOne</code>	User and Profile	Share primary key with <code>@MapsId</code>

Type	Annotation	Example	Notes
Many-to-Many	<code>@ManyToMany</code> + <code>@JoinTable</code>	Task and Tag	Prefer an explicit join entity if it has extra columns

JAVA

```

@Entity
public class AppUser {
    @Id @GeneratedValue private Long id;

    @OneToMany(mappedBy = "owner", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Task> tasks = new ArrayList<>();

    public void addTask(Task t) { tasks.add(t); t.setOwner(this); } // keep both sides in sync
}

```

5.4 Spring Data JPA repositories

JAVA

```

public interface TaskRepository extends JpaRepository<Task, Long> {

    // 1) Derived queries: method name becomes SQL
    List<Task> findByStatusOrderByDueDateAsc(Task.Status status);
    Optional<Task> findByIdAndOwnerId(Long id, Long ownerId);
    boolean existsByTitleIgnoreCase(String title);
    long countByStatus(Task.Status status);
    // 2) JPQL with named parameters
    @Query("select t from Task t where t.owner.id = :ownerId and t.dueDate < :today and t.status <> 'DONE'")
    List<Task> findOverdue(@Param("ownerId") Long ownerId, @Param("today") LocalDate today);

    // 3) Native SQL
    @Query(value = "select * from tasks where title ilike %:q%", nativeQuery = true)
    List<Task> search(@Param("q") String q);
    // 4) Pagination + sorting
    Page<Task> findByOwnerId(Long ownerId, Pageable pageable);

    // 5) Fetch join to avoid the N+1 problem
    @Query("select t from Task t join fetch t.owner where t.status = :s")
    List<Task> findWithOwner(@Param("s") Task.Status s);

    // 6) Modifying query
    @Modifying @Query("update Task t set t.status = 'DONE' where t.id in :ids")
    int markDone(@Param("ids") List<Long> ids);

    // 7) DTO projection (only fetch needed columns)
    record TaskSummary(Long id, String title) {}
    List<TaskSummary> findSummariesByStatus(Task.Status status);
}

```

JAVA

```

// Using pagination
Pageable p = PageRequest.of(0, 20, Sort.by("dueDate").descending());
Page<Task> page = repo.findByOwnerId(7L, p);
page.getContent(); page.getTotalElements(); page.getTotalPages();

```

WARNING

The N+1 problem: loading 100 tasks then touching `task.getOwner().getName()` fires 1 + 100 queries. Fix with `join fetch`, `@EntityGraph(attributePaths = "owner")`, or batch fetching (`hibernate.default_batch_fetch_size`).

5.5 Service layer with transactions

JAVA

```

@Service
@Transactional(readOnly = true)           // default for all methods (read-only optimisation)
public class TaskService {
    private final TaskRepository repo;
    private final UserRepository users;
    private final TaskMapper mapper;

    public TaskService(TaskRepository repo, UserRepository users, TaskMapper mapper) {
        this.repo = repo; this.users = users; this.mapper = mapper;
    }

    public TaskResponse get(Long id) {
        return repo.findById(id).map(mapper::toResponse)
            .orElseThrow(() -> new NotFoundException("Task " + id + " not found"));
    }

    @Transactional                        // write transaction
    public TaskResponse create(CreateTask dto, Long ownerId) {
        AppUser owner = users.getReferenceById(ownerId);
        Task saved = repo.save(new Task(dto.title(), owner));
        return mapper.toResponse(saved);
    }

    @Transactional
    public void complete(Long id) {
        Task t = repo.findById(id).orElseThrow(() -> new NotFoundException("Task " + id));
        t.setStatus(Task.Status.DONE);           // dirty checking: UPDATE runs at commit, no save() needed
    }
}

```

Transaction rules to remember

- Rollback happens by default on **unchecked** exceptions (RuntimeException, Error). For checked exceptions use `@Transactional(rollbackFor = Exception.class)`.
- `@Transactional` works through a **proxy**. Calling a transactional method from another method **in the same class** (`this.other()`) bypasses the proxy, so no transaction starts.
- `@Transactional` on private methods is ignored.
- Propagation: **REQUIRED** (default, join existing), **REQUIRES_NEW** (always new, suspends outer), **MANDATORY**, **SUPPORTS**, **NEVER**.
- Isolation levels: **READ_COMMITTED** (most DB default), **REPEATABLE_READ**, **SERIALIZABLE**.

5.6 DTOs and mapping

JAVA

```

public record TaskResponse(Long id, String title, String status, LocalDate dueDate) {}

@Mapper(componentModel = "spring")           // MapStruct generates the implementation at compile time
public interface TaskMapper {
    @Mapping(target = "status", expression = "java(task.getStatus().name())")
    TaskResponse toResponse(Task task);
}

```

MapStruct is generated code (no reflection), so it is fast and type-safe. Alternatives: ModelMapper (reflection, slower) or hand-written mappers.

5.7 Controller with validation, pagination and proper status codes

JAVA

```

@RestController
@RequestMapping("/api/tasks")
@Validated
public class TaskController {
    private final TaskService service;
    public TaskController(TaskService service) { this.service = service; }

    @GetMapping("/{id}")
    public TaskResponse get(@PathVariable Long id) { return service.get(id); }

    @GetMapping
    public Page<TaskResponse> list(@RequestParam(defaultValue = "0") @Min(0) int page,
                                   @RequestParam(defaultValue = "20") @Max(100) int size) {
        return service.list(PageRequest.of(page, size));
    }
    @PostMapping
    public ResponseEntity<TaskResponse> create(@Valid @RequestBody CreateTask dto,
                                                UriComponentsBuilder uri) {
        TaskResponse created = service.create(dto, currentUserId());
        URI location = uri.path("/api/tasks/{id}").build(created.id());
        return ResponseEntity.created(location).body(created); // 201 + Location header
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}

```

5.8 Global exception handling (RFC 9457 Problem Details)

JAVA

```

public class NotFoundException extends RuntimeException {
    public NotFoundException(String msg) { super(msg); }
}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    ProblemDetail notFound(NotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Resource not found");
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail invalid(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Validation failed");
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(f -> errors.put(f.getField(), f.getDefaultMessage()));
        pd.setProperty("errors", errors);
        return pd;
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail fallback(Exception ex) {
        log.error("Unhandled", ex);
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    }
}

```

Enable Problem Details for framework-level errors with `spring.mvc.problemdetails.enabled=true`.

5.9 Database migrations

Never rely on `ddl-auto=update` in production. Use **Flyway** or **Liquibase**:

SQL

```
-- src/main/resources/db/migration/V1__create_tasks.sql
create table tasks (
  id bigserial primary key,
  title varchar(100) not null,
  status varchar(20) not null default 'OPEN',
  due_date date,
  owner_id bigint not null references app_user(id),
  version bigint not null default 0,
  created_at timestamptz not null default now()
);
create index idx_tasks_owner_status on tasks(owner_id, status);
```

Add **spring-boot-starter-flyway** (Boot 4) or **flyway-core** (Boot 3) and Boot runs pending migrations at startup.

6. Node.js - Intermediate

CATEGORY

Building real applications. Express architecture, middleware, validation with Zod, Prisma/Drizzle/TypeORM, error handling, transactions, pagination and NestJS structure. Level: Intermediate.

6.1 Recommended project structure

TEXT

```
taskapi/
  src/
    index.ts          <- starts the server
    app.ts            <- builds the Express app (easy to test)
    config/env.ts
    modules/
      tasks/
        task.routes.ts <- routes
        task.controller.ts <- HTTP layer
        task.service.ts <- business logic
        task.repo.ts    <- database access
        task.schema.ts  <- zod schemas + types
      middleware/ (auth.ts, error.ts, validate.ts)
      lib/ (db.ts, logger.ts)
    prisma/schema.prisma
    tests/
```

6.2 Middleware: the heart of Express

Middleware is a function (**req**, **res**, **next**). Requests flow through a **pipeline**; each function can modify the request, end the response, or call **next()**.

TS

```
// Logging middleware
app.use((req, _res, next) => {
  console.log(req.method, req.url);
  next(); // pass control on
});

// Route-level middleware
router.post("/", requireAuth, validate(CreateTask), controller.create);
// A reusable validation middleware factory
export const validate = (schema: z.ZodTypeAny) => {
  (req: Request, res: Response, next: NextFunction) => {
    const r = schema.safeParse(req.body);
    if (!r.success) return res.status(400).json({ errors: r.error.flatten().fieldErrors });
    req.body = r.data; // replaced with parsed + typed data
    next();
  };
};
```

Spring equivalent	Express equivalent
Filter / OncePerRequestFilter	<code>app.use(fn)</code>
<code>HandlerInterceptor.preHandle</code>	Route-level middleware
<code>@RestControllerAdvice</code>	Error middleware (4 params)
<code>HandlerMethodArgumentResolver</code>	Middleware that attaches to <code>req</code>

6.3 Router, controller, service

TS

```
// task.routes.ts
import { Router } from "express";
export const taskRouter = Router();
taskRouter.get("/", ctrl.list);
taskRouter.get("/:id", ctrl.get);
taskRouter.post("/", requireAuth, validate(CreateTask), ctrl.create);
taskRouter.delete("/:id", requireAuth, ctrl.remove);
// app.ts
app.use("/api/tasks", taskRouter);
```

TS

```
// task.controller.ts
export const get = async (req: Request, res: Response) => {
  const task = await taskService.get(Number(req.params.id));
  res.json(task);
};
// Express 5 forwards rejected promises to the error handler automatically.
// In Express 4 wrap handlers: const wrap = fn => (req, res, next) => fn(req, res, next).catch(next);
```

TS

```
// task.service.ts
export class NotFoundError extends Error {
  status = 404;
  constructor(msg: string) { super(msg); }
}

export const taskService = {
  async get(id: number) {
    const task = await prisma.task.findUnique({ where: { id } });
    if (!task) throw new NotFoundError(`Task ${id} not found`);
    return task;
  },
};
```

6.4 Centralised error handling

TS

```
export function errorHandler(err: any, _req: Request, res: Response, _next: NextFunction) {
  if (err instanceof z.ZodError) {
    return res.status(400).json({ title: "Validation failed", errors: err.flatten() });
  }
  const status = err.status ?? 500;
  if (status >= 500) logger.error({ err }, "unhandled");
  res.status(status).json({ title: status === 500 ? "Internal Server Error" : err.message });
}
app.use(errorHandler); // must be registered LAST

// Also protect the process:
process.on("unhandledRejection", (r) => { logger.fatal({ r }); process.exit(1); });
process.on("uncaughtException", (e) => { logger.fatal({ e }); process.exit(1); });
```

6.5 Database access: ORM comparison

Tool	Style	Type safety	Strengths	Weaknesses
Prisma	Schema file + generated client	Excellent	Best DX, migrations, readable queries	Heavier engine, complex raw SQL is awkward
Drizzle	SQL-like TypeScript	Excellent	Lightweight, close to SQL, edge-friendly	Younger, fewer guides
TypeORM	Decorators (like JPA)	Good	Familiar to Java developers, Active Record + Data Mapper	Known edge-case bugs, slower evolution

Tool	Style	Type safety	Strengths	Weaknesses
Sequelize	Classic ORM	Fair	Mature	TypeScript support is clunky
Knex	Query builder	Manual	Flexible SQL building	No entity mapping
Mongoose	ODM for MongoDB	Good	Schemas and hooks for Mongo	MongoDB only

PRISMA

```
// prisma/schema.prisma
model Task {
  id      Int      @id @default(autoincrement())
  title   String    @db.VarChar(100)
  status  Status    @default(OPEN)
  dueDate DateTime?
  owner   User      @relation(fields: [ownerId], references: [id])
  ownerId Int
  createdAt DateTime @default(now())
  @@index([ownerId, status])
}
enum Status { OPEN IN_PROGRESS DONE }
```

TS

```
// Prisma client usage (compare with Spring Data JPA)
const tasks = await prisma.task.findMany({
  where: { ownerId: 7, status: "OPEN" },
  orderBy: { dueDate: "asc" },
  include: { owner: { select: { name: true } } }, // like join fetch
  skip: 0, take: 20,                             // pagination
});
const created = await prisma.task.create({ data: { title: "Learn", ownerId: 7 } });
await prisma.task.update({ where: { id: 1 }, data: { status: "DONE" } });

// Transaction: all-or-nothing
await prisma.$transaction(async (tx) => {
  const t = await tx.task.update({ where: { id: 1 }, data: { status: "DONE" } });
  await tx.auditLog.create({ data: { taskId: t.id, action: "COMPLETED" } });
});
```

6.6 Spring Data JPA vs Prisma (mapping)

Task	Spring Data JPA	Prisma
Find by id	<code>repo.findById(1L)</code>	<code>prisma.task.findUnique({ where: { id: 1 } })</code>
Find all with filter	<code>repo.findByStatus(OPEN)</code>	<code>prisma.task.findMany({ where: { status: "OPEN" } })</code>
Save	<code>repo.save(entity)</code>	<code>prisma.task.create({ data })</code>
Update	Change entity in a transaction (dirty check)	<code>prisma.task.update({ where, data })</code>
Delete	<code>repo.deleteById(1L)</code>	<code>prisma.task.delete({ where: { id: 1 } })</code>
Pagination	<code>PageRequest.of(0, 20)</code>	<code>skip</code> and <code>take</code>
Join	<code>join fetch</code> / <code>@EntityGraph</code>	<code>include: { owner: true }</code>
Transaction	<code>@Transactional</code>	<code>prisma.\$transaction(...)</code>
Migrations	Flyway / Liquibase	<code>prisma migrate dev / deploy</code>
Lazy loading	Yes (source of N+1)	No - explicit <code>include</code> only

6.7 NestJS: Spring-style structure for Node

TS

```
// task.module.ts
@Module({
  controllers: [TaskController],
  providers: [TaskService, PrismaService],
  exports: [TaskService],
})
export class TaskModule {}

// task.service.ts
@Injectable()
export class TaskService {
  constructor(private readonly prisma: PrismaService) {} // constructor DI, like Spring
  async get(id: number) {
    const t = await this.prisma.task.findUnique({ where: { id } });
    if (!t) throw new NotFoundException(`Task ${id} not found`); // -> 404 automatically
    return t;
  }
}

// task.controller.ts
@Controller("tasks")
export class TaskController {
  constructor(private readonly service: TaskService) {}
  @Get(":id")
  get(@Param("id", ParseIntPipe) id: number) { return this.service.get(id); }

  @Post()
  @UseGuards(JwtAuthGuard)
  create(@Body() dto: CreateTaskDto) { return this.service.create(dto); }
}

// main.ts
const app = await NestFactory.create(AppModule);
app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
await app.listen(3000);
```

NestJS building block	Spring equivalent
Module	@Configuration / package + component scan
Controller	@RestController
Provider / Injectable	@Service / @Component
Pipe	Argument resolver + @Valid
Guard	Spring Security filter / @PreAuthorize
Interceptor	HandlerInterceptor / AOP @Around
Exception filter	@ControllerAdvice
ConfigModule	@ConfigurationProperties

6.7.1 Pagination patterns (both stacks)

Pattern	How it works	Pros	Cons
Offset / page	LIMIT 20 OFFSET 40	Simple, jump to page N	Slow on huge tables, unstable if data changes
Keyset / cursor	WHERE id > :lastId ORDER BY id LIMIT 20	Fast at any depth, stable	No random page jump

PRO TIP

For infinite scroll and large datasets always prefer **keyset (cursor) pagination**.

7. Security and Authentication

CATEGORY

Cross-cutting concern. Authentication (who are you?), authorisation (what may you do?), JWT, password hashing, CORS, CSRF and hardening in both stacks. Level: Intermediate to Advanced.

7.1 Core concepts

Term	Meaning
Authentication (AuthN)	Proving identity (login with password, OAuth, passkey)
Authorisation (AuthZ)	Deciding what an identity may do (roles, permissions)
Session	Server remembers you via a session id cookie (stateful)
JWT	Signed token containing claims; server verifies signature (stateless)
OAuth 2.0 / OIDC	Delegated login ("Sign in with Google"); OIDC adds identity on top
CORS	Browser rule controlling which origins may call your API
CSRF	Attack that abuses cookies; needs tokens or SameSite cookies
Hashing	One-way password storage: bcrypt / Argon2 / scrypt (never plain SHA-256)

7.2 JWT structure

TEXT

```
header.payload.signature      (each part Base64URL encoded)

header  = {"alg":"HS256","typ":"JWT"}
payload = {"sub":"42","roles":["USER"],"iat":1719000000,"exp":1719000900}
signature= HMAC_SHA256(header + "." + payload, secret)
```

WARNING

A JWT is **signed, not encrypted**. Anyone can read the payload. Never put secrets in it. Keep access tokens short (5-15 min) and use refresh tokens (stored in an httpOnly, Secure, SameSite cookie).

7.3 Spring Security (6.x / 7.x) with JWT resource server

Spring Security is a **chain of servlet filters**. Requests pass through the `SecurityFilterChain` before reaching controllers.

JAVA

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity                // enables @PreAuthorize
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())           // OK for stateless token APIs (no cookies for auth)
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers("/api/auth/**", "/actuator/health", "/v3/api-docs/**").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults())); // validates Bearer JWT
    }
}
```

```

        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration c = new CorsConfiguration();
        c.setAllowedOrigins(List.of("https://app.example.com"));
        c.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE"));
        c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        UrlBasedCorsConfigurationSource s = new UrlBasedCorsConfigurationSource();
        s.registerCorsConfiguration("/**", c);
        return s;
    }
}

```

YAML

```

# application.yml - validate tokens issued by your identity provider (Keycloak, Auth0, Cognito...)
spring.security.oauth2.resourceserver.jwt.issuer-uri: https://auth.example.com/realms/main

```

JAVA

```

// Method-level authorisation
@PreAuthorize("hasRole('ADMIN') or #ownerId == authentication.principal.claims['sub']")
public List<Task> byOwner(String ownerId) { ... }

// Get the current user in a controller
@GetMapping("/me")
public Map<String, Object> me(@AuthenticationPrincipal Jwt jwt) {
    return Map.of("id", jwt.getSubject(), "roles", jwt.getClaimAsStringList("roles"));
}

```

7.4 Node.js: JWT auth with Express and jose

TS

```

import { SignJWT, jwtVerify } from "jose";
import bcrypt from "bcrypt";

const secret = new TextEncoder().encode(env.JWT_SECRET); // at least 32 random bytes

export async function login(req: Request, res: Response) {
    const { email, password } = req.body;
    const user = await prisma.user.findUnique({ where: { email } });
    const ok = user && await bcrypt.compare(password, user.passwordHash);
    if (!ok) return res.status(401).json({ title: "Invalid credentials" }); // same message for both cases

    const token = await new SignJWT({ roles: user.roles })
        .setProtectedHeader({ alg: "HS256" })
        .setSubject(String(user.id))
        .setIssuedAt()
        .setExpirationTime("15m")
        .sign(secret);
    res.json({ accessToken: token });
}

export async function requireAuth(req: Request, res: Response, next: NextFunction) {
    const h = req.header("authorization");
    if (!h?.startsWith("Bearer ")) return res.status(401).end();
    try {
        const { payload } = await jwtVerify(h.slice(7), secret);
        (req as any).user = { id: payload.sub, roles: payload.roles as string[] };
        next();
    } catch { res.status(401).json({ title: "Invalid or expired token" }); }
}

export const requireRole = (role: string) => (req: Request, res: Response, next: NextFunction) =>
    (req as any).user?.roles?.includes(role) ? next() : res.status(403).end();
// usage
router.delete("/:id", requireAuth, requireRole("ADMIN"), ctrl.remove);

```

TS

```
// Hardening middleware (do all of these)
import helmet from "helmet";
import cors from "cors";
import rateLimit from "express-rate-limit";

app.use(helmet()); // secure HTTP headers
app.use(cors({ origin: ["https://app.example.com"], credentials: true }));
app.use(rateLimit({ windowMs: 60_000, limit: 100 })); // 100 req/min per IP
app.use(express.json({ limit: "100kb" })); // cap body size
app.disable("x-powered-by");
```

7.5 Security comparison

Concern	Spring Boot	Node.js
Framework support	Spring Security: comprehensive, opinionated, complex	DIY with libraries (Passport, jose, Helmet), or NestJS guards
Password hashing	<code>BCryptPasswordEncoder</code> , <code>Argon2PasswordEncoder</code>	<code>bcrypt</code> , <code>argon2</code> packages
Method-level rules	<code>@PreAuthorize</code> , <code>@Secured</code>	Custom middleware / NestJS <code>@Roles()</code> guard
OAuth2 / OIDC	First-class (client + resource server + auth server)	Passport strategies, <code>openid-client</code> , Auth.js, Keycloak adapters
CSRF protection	On by default for browser sessions	Manual (<code>csurf</code> alternatives) or SameSite cookies
Security headers	Added by default	Add <code>helmet</code> yourself
Dependency risk	Fewer, larger dependencies	Deep npm trees: audit constantly (<code>npm audit</code> , Snyk)

7.6 OWASP-style checklist (applies to both)

1. Validate **every** input on the server (never trust the client).
2. Use parameterised queries / ORM (prevents SQL injection). Never build SQL with string concatenation.
3. Hash passwords with `bcrypt`/`Argon2`; never log passwords or tokens.
4. Return the same error for "wrong email" and "wrong password".
5. Apply rate limiting and account lockout on login endpoints.
6. Use HTTPS everywhere; set `Secure`, `HttpOnly`, `SameSite` on cookies.
7. Restrict CORS to known origins (never `*` with credentials).
8. Keep secrets in environment variables / a vault, not in Git.
9. Do authorisation checks **on the object** (is this task owned by this user?), not just on the route (broken object-level authorisation is the #1 API risk).
10. Keep dependencies updated; scan them in CI.

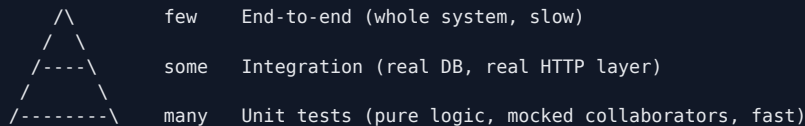
8. Testing

CATEGORY

Quality engineering. Unit, slice, integration and end-to-end testing with mocks and real databases in containers. Level: Intermediate.

8.1 The test pyramid

TEXT



8.2 Spring Boot testing

JAVA

```
// 1) Pure unit test (JUnit 5 + Mockito) - no Spring context, very fast
@ExtendWith(MockitoExtension.class)
class TaskServiceTest {
    @Mock TaskRepository repo;
    @Mock UserRepository users;
    @Mock TaskMapper mapper;
    @InjectMocks TaskService service;
    @Test
    void get_throwsWhenMissing() {
        when(repo.findById(99L)).thenReturn(Optional.empty());
        assertThatThrownBy(() -> service.get(99L))
            .isInstanceOf(NotFoundException.class)
            .hasMessageContaining("99");
    }
}
```

JAVA

```
// 2) Web slice test - only controllers + Jackson + validation are loaded
@WebMvcTest(TaskController.class)
class TaskControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean TaskService service; // @MockBean in Boot 3.x

    @Test
    @WithMockUser
    void create_returns201() throws Exception {
        when(service.create(any(), anyLong())).thenReturn(new TaskResponse(1L, "Learn", "OPEN", null));
        mvc.perform(post("/api/tasks").with(csrf())
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"title":"Learn","priority":3}
                """))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location", endsWith("/api/tasks/1")))
            .andExpect(jsonPath("$.title").value("Learn"));
    }

    @Test
    @WithMockUser
    void create_returns400WhenTitleBlank() throws Exception {
        mvc.perform(post("/api/tasks").with(csrf()).contentType(MediaType.APPLICATION_JSON)
            .content("{\"title\":\"\", \"priority\":3}"))
            .andExpect(status().isBadRequest());
    }
}
```

JAVA

```
// 3) Repository / integration test with a REAL PostgreSQL in Docker (Testcontainers)
@DataJpaTest
@Testcontainers
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class TaskRepositoryIT {
    @Container @ServiceConnection // Boot 3.1+: wires datasource automatically
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16");
    @Autowired TaskRepository repo;

    @Test
    void findOverdue_returnsOnlyOpenPastDue() { /* arrange, act, assert */ }
}

// 4) Full application test
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class ApiIT { @Autowired TestRestTemplate rest; }
```

Annotation	Loads	Speed
none (Mockito)	Nothing	Fastest
@WebMvcTest	Web layer only	Fast
@DataJpaTest	JPA + repositories + embedded/real DB	Medium
@JsonTest	Jackson only	Fast
@SpringBootTest	Everything	Slowest

8.3 Node.js testing with Vitest and Supertest

TS

```
// task.service.test.ts - unit test with a mocked Prisma client
import { describe, it, expect, vi, beforeEach } from "vitest";
import { taskService, NotFoundError } from "../task.service";
import { prisma } from "../../lib/db";

vi.mock("../../lib/db", () => ({ prisma: { task: { findUnique: vi.fn() } } }));

describe("taskService.get", () => {
    beforeEach(() => vi.clearAllMocks());

    it("throws NotFoundError when missing", async () => {
        vi.mocked(prisma.task.findUnique).mockResolvedValue(null);
        await expect(taskService.get(99)).rejects.toBeInstanceOf(NotFoundError);
    });

    it("returns the task", async () => {
        vi.mocked(prisma.task.findUnique).mockResolvedValue({ id: 1, title: "Learn" } as any);
        await expect(taskService.get(1)).resolves.toMatchObject({ title: "Learn" });
    });
});
```

TS

```
// task.api.test.ts - HTTP-level test (no port needed because app is exported separately)
import request from "supertest";
import { app } from "../app";

describe("POST /api/tasks", () => {
  it("returns 400 for invalid body", async () => {
    const res = await request(app).post("/api/tasks")
      .set("Authorization", `Bearer ${token}`)
      .send({ title: "" });
    expect(res.status).toBe(400);
    expect(res.body.errors.title).toBeDefined();
  });

  it("creates a task", async () => {
    const res = await request(app).post("/api/tasks")
      .set("Authorization", `Bearer ${token}`)
      .send({ title: "Learn", priority: 3 });
    expect(res.status).toBe(201);
    expect(res.body).toMatchObject({ title: "Learn" });
  });
});
```

TS

```
// Real database tests with Testcontainers (Node version)
import { PostgreSQLContainer } from "@testcontainers/postgresql";
let container;
beforeAll(async () => {
  container = await new PostgreSQLContainer("postgres:16").start();
  process.env.DATABASE_URL = container.getConnectionUri();
  // run prisma migrate deploy here
}, 60_000);
afterAll(() => container.stop());
```

8.4 Testing comparison

Aspect	Spring Boot	Node.js
Test runner	JUnit 5	Vitest / Jest / <code>node:test</code>
Mocking	Mockito, <code>@MockitoBean</code>	<code>vi.mock</code> , <code>jest.mock</code> , <code>sinon</code>
Assertions	AssertJ, Hamcrest	<code>expect(...)</code> (Jest-style), <code>node:assert</code>
HTTP testing	MockMvc, WebTestClient	Supertest, <code>app.inject()</code> (Fastify)
Real DB	Testcontainers + <code>@ServiceConnection</code>	Testcontainers for Node
Slice tests	<code>@WebMvcTest</code> , <code>@DataJpaTest</code> (built in)	Compose your own
Coverage	JaCoCo	c8 / istanbul (built into Vitest and Jest)
Speed	Context startup can be slow (cache contexts)	Very fast startup and watch mode
Snapshot tests	Rare	Common (<code>toMatchSnapshot</code>)

PRO TIP

Test **behaviour** (inputs and outputs), not implementation details. Prefer one real database in a container over many mocks for repository tests: mocks cannot catch SQL mistakes.

9. Spring Boot - Advanced

CATEGORY

Production-grade Spring. AOP, caching, async, scheduling, events, reactive WebFlux, virtual threads, resilience, messaging, observability, native images and containerisation. Level: Advanced.

9.1 Caching

JAVA

```
@Configuration @EnableCaching
class CacheConfig {}

@Service
class ProductService {
    @Cacheable(cacheNames = "products", key = "#id")           // read-through
    public Product get(Long id) { return repo.findById(id).orElseThrow(); }
    @CachePut(cacheNames = "products", key = "#result.id")    // always run, update cache
    public Product update(Product p) { return repo.save(p); }

    @CacheEvict(cacheNames = "products", key = "#id")        // remove entry
    public void delete(Long id) { repo.deleteById(id); }
}
```

Use **Caffeine** for in-process caches (fast, TTL, max size) and **Redis** ([spring-boot-starter-data-redis](#)) for shared caches across instances.

YAML

```
spring:
  cache:
    type: caffeine
    caffeine.spec: maximumSize=10000,expireAfterWrite=10m
```

9.2 Async, scheduling and events

JAVA

```
@EnableAsync @EnableScheduling @SpringBootApplication
class App {}

@Service
class ReportService {
    @Async                                           // runs on a task executor thread
    public CompletableFuture<Report> build(Long id) { return CompletableFuture.completedFuture(compute(id)); }
    @Scheduled(cron = "0 0 2 * * *")              // every day at 02:00
    void nightlyCleanup() { /* ... */ }

    @Scheduled(fixedDelayString = "PT30S")         // 30 s after previous run finished
    void poll() { /* ... */ }
}

// Domain events: decouple modules inside one application
record TaskCompleted(Long taskId) {}

@Service class TaskService {
    private final ApplicationEventPublisher events;
    void complete(Long id) { /* ... */ events.publishEvent(new TaskCompleted(id)); }
}

@Component class NotificationListener {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) // only if DB commit succeeded
    void on(TaskCompleted e) { /* send email */ }
}
```

WARNING

With multiple app instances `@Scheduled` runs on **every** instance. Use **ShedLock** or Quartz clustering for single-run jobs.

9.3 AOP (Aspect-Oriented Programming)

Cross-cutting logic (logging, timing, auditing) without touching business code.

JAVA

```
@Aspect @Component
class TimingAspect {
    @Around("@annotation(Timed) || execution(* com.example.taskapi.service..*(..))")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long t0 = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - t0) / 1_000_000); }
    }
}
```

`@Transactional`, `@Cacheable`, `@Async` and `@PreAuthorize` are all implemented with AOP proxies. This explains the self-invocation pitfall.

9.4 Virtual threads (Java 21+) - the game changer

YAML

```
spring.threads.virtual.enabled: true      # Boot 3.2+: Tomcat and @Async use virtual threads
```

With virtual threads, blocking code (JDBC, RestClient) no longer wastes an expensive OS thread while waiting. You get **event-loop-like scalability while keeping simple imperative code**. This closes much of Node.js's traditional advantage for I/O-heavy workloads.

Caveat	Detail
Pinning	<code>synchronized</code> blocks that do blocking I/O can pin the carrier thread (improved in newer JDKs; prefer <code>ReentrantLock</code>)
Pool limits	The DB connection pool (e.g. Hikari 10-20) is still the real bottleneck; virtual threads do not create connections
ThreadLocal	Works, but avoid heavy per-thread state

9.5 Reactive stack: Spring WebFlux

JAVA

```
@RestController
class PriceController {
    private final WebClient client = WebClient.create("https://api.example.com");

    @GetMapping("/prices")
    Flux<Price> prices() {
        return client.get().uri("/prices").retrieve().bodyToFlux(Price.class) // non-blocking
            .timeout(Duration.ofSeconds(2))
            .onErrorResume(e -> Flux.empty());
    }

    @GetMapping("/user/{id}")
    Mono<User> user(@PathVariable String id) { // Mono = 0..1 items, Flux = 0..N
        return users.findById(id).switchIfEmpty(Mono.error(new NotFoundException(id)));
    }
}
```

	Spring MVC (+ virtual threads)	Spring WebFlux
Programming model	Imperative, blocking style	Reactive streams (Mono/Flux)

	Spring MVC (+ virtual threads)	Spring WebFlux
Debuggability	Easy (normal stack traces)	Hard
Database	JDBC / JPA	R2DBC, reactive Mongo/Redis
Use when	90% of applications	Streaming, backpressure, very high concurrency, gateways

PRO TIP

In 2024+ the default advice is: **start with Spring MVC + virtual threads**; choose WebFlux only for genuinely streaming or reactive end-to-end workloads.

9.6 HTTP clients and resilience

JAVA

```
// RestClient (Boot 3.2+): modern synchronous client, replaces RestTemplate
RestClient rc = RestClient.builder().baseUrl("https://api.example.com").build();
Weather w = rc.get().uri("/weather?city={c}", "Chennai").retrieve().body(Weather.class);

// Declarative HTTP interface (like Feign)
@HttpExchange("/weather")
interface WeatherApi { @GetExchange Weather byCity(@RequestParam String city); }
```

JAVA

```
// Resilience4j: circuit breaker, retry, rate limiter, bulkhead
@CircuitBreaker(name = "payments", fallbackMethod = "fallback")
@Retry(name = "payments")
@TimeLimiter(name = "payments")
public PaymentResult pay(PaymentRequest r) { return paymentClient.charge(r); }
PaymentResult fallback(PaymentRequest r, Throwable t) { return PaymentResult.pending(); }
```

Circuit breaker states: **CLOSED** (normal) -> **OPEN** (failing fast after threshold) -> **HALF-OPEN** (probe a few calls) -> CLOSED.

9.7 Messaging with Kafka

JAVA

```
@Service
class OrderProducer {
    private final KafkaTemplate<String, OrderCreated> kafka;
    OrderProducer(KafkaTemplate<String, OrderCreated> kafka) { this.kafka = kafka; }
    void publish(OrderCreated e) {
        kafka.send("orders.created", e.orderId(), e); // key = ordering per order
    }
}

@Component
class OrderConsumer {
    @KafkaListener(topics = "orders.created", groupId = "billing")
    void handle(OrderCreated e, Acknowledgment ack) {
        // must be IDEMPOTENT: messages can be delivered more than once
        billing.process(e);
        ack.acknowledge();
    }
}
```

Reliability patterns: **Outbox pattern** (write event to an outbox table in the same DB transaction, publish afterwards), **idempotent consumers**, **dead-letter topics**, **exactly-once semantics** only where needed.

9.8 Observability with Actuator, Micrometer and OpenTelemetry

YAML

```
management:
  endpoints.web.exposure.include: health,info,prometheus
  endpoint.health.probes.enabled: true           # /actuator/health/liveness and /readiness for Kubernetes
  tracing.sampling.probability: 0.1
```

JAVA

```
@Autowired MeterRegistry registry;
Counter c = Counter.builder("tasks.created").tag("source", "api").register(registry);
c.increment();

@Observed(name = "invoice.generate")           // creates metric + trace span automatically
public Invoice generate(Long id) { ... }
```

The three pillars: **logs** (structured JSON with trace id), **metrics** (Prometheus + Grafana), **traces** (OpenTelemetry to Tempo/Jaeger).

9.9 Performance tuning checklist

1. Fix **N+1 queries** (`join fetch`, `@EntityGraph`, batch size). Turn on `spring.jpa.show-sql` only locally.
2. Set `open-in-view: false`; use DTO projections for read endpoints.
3. Size the **Hikari** connection pool (`maximum-pool-size` about 10-20 for most services, not hundreds).
4. Add the right **database indexes** (check with `EXPLAIN ANALYZE`).
5. Use **caching** for hot read-mostly data (Caffeine or Redis).
6. Enable **HTTP compression**, and **virtual threads** when I/O-bound.
7. JVM: container-aware defaults, `-XX:MaxRAMPercentage=75`, G1 or ZGC, and measure with JFR (Java Flight Recorder).
8. Use **batch inserts** (`saveAll` with `hibernate.jdbc.batch_size`) and avoid `IDENTITY` ids when batching heavily (prefer `SEQUENCE`).

9.10 Native images and containers

BASH

```
# Layered JAR image via Cloud Native Buildpacks (no Dockerfile needed)
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=taskapi:1.0

# GraalVM native image: startup in milliseconds, memory ~ 5-10x smaller, longer build time
./mvnw -Pnative native:compile
```

DOCKERFILE

```
# Dockerfile (multi-stage, JRE only at runtime)
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
USER 1000
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

9.11 Microservices with Spring

Concern	Tool
API gateway	Spring Cloud Gateway
Service discovery	Kubernetes DNS (preferred) or Eureka / Consul
Config	Kubernetes ConfigMaps/Secrets, Spring Cloud Config, Vault
Sync calls	RestClient, HTTP Interface, gRPC
Async calls	Kafka, RabbitMQ
Distributed tracing	Micrometer Tracing + OpenTelemetry
Data consistency	Saga pattern, Outbox pattern

NOTE

Do not start with microservices. Begin with a **modular monolith** (Spring Modulith helps enforce module boundaries) and split only when a clear scaling or team-autonomy need appears.

10. Node.js - Advanced

CATEGORY

Production-grade Node. Streams, worker threads, clustering, performance and memory, queues, WebSockets, Fastify, graceful shutdown, observability and deployment. Level: Advanced.

10.1 Streams and backpressure

Streams process data **chunk by chunk** so memory stays flat even for multi-GB files.

TS

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";
import { createGzip } from "node:zlib";
import { Transform } from "node:stream";

const upper = new Transform({
  transform(chunk, _enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});
await pipeline(
  createReadStream("big.log"),
  upper,
  createGzip(),
  createWriteStream("big.log.gz"),
);
// Stream a large download from an Express route
app.get("/export", async (_req, res) => {
  res.setHeader("Content-Type", "text/csv");
  await pipeline(createReadStream("export.csv"), res);
});
```

10.2 Worker threads for CPU-bound work

TS

```
// main.ts
import { Worker } from "node:worker_threads";

function hashInWorker(data: string): Promise<string> {
  return new Promise((resolve, reject) => {
    const w = new Worker(new URL("./hash-worker.js", import.meta.url), { workerData: data });
    w.once("message", resolve);
    w.once("error", reject);
  });
}

// hash-worker.js
import { parentPort, workerData } from "node:worker_threads";
import { scryptSync } from "node:crypto";
parentPort!.postMessage(scryptSync(workerData, "salt", 64).toString("hex"));
```

In production use a **worker pool** ([piscina](#)) so you do not spawn a thread per request.

10.3 Scaling across CPU cores

Approach	How	Notes
cluster module	Fork one process per core sharing a port	Built in; stateless apps only
PM2	<code>pm2 start dist/index.js -i max</code>	Process manager with restarts and logs
Containers	Run N replicas behind a load balancer (Kubernetes)	Best practice today; one process per container
Worker threads	Parallelise CPU tasks inside one process	For compute, not for scaling HTTP

WARNING

Keep servers **stateless**. Sessions, caches and websocket state must live in Redis/DB, otherwise scaling to several instances breaks.

10.4 Fastify: performance-oriented alternative

TS

```
import Fastify from "fastify";
import { z } from "zod";
import { serializerCompiler, validatorCompiler, ZodTypeProvider } from "fastify-type-provider-zod";

const app = Fastify({ logger: true }).withTypeProvider<ZodTypeProvider>();
app.setValidatorCompiler(validatorCompiler);
app.setSerializerCompiler(serializerCompiler);
app.get("/tasks/:id", {
  schema: {
    params: z.object({ id: z.coerce.number() }),
    response: { 200: z.object({ id: z.number(), title: z.string() }) }, // fast, safe serialisation
  },
}, async (req) => {
  return taskService.get(req.params.id); // req.params.id is typed as number
});
app.addHook("onRequest", async (req) => { /* auth */ }); // hooks instead of middleware
await app.listen({ port: 3000, host: "0.0.0.0" });
```

Fastify advantages: schema-based validation and serialisation (often noticeably faster than Express), encapsulated plugins, built-in logging (pino), `app.inject()` for tests.

10.5 Background jobs and queues (BullMQ + Redis)

TS

```
import { Queue, Worker } from "bullmq";
const connection = { host: "redis", port: 6379 };

export const emailQueue = new Queue("emails", { connection });
await emailQueue.add("welcome", { to: "a@b.com" }, {
  attempts: 5, backoff: { type: "exponential", delay: 2000 } });

new Worker("emails", async (job) => { await sendEmail(job.data.to); }, {
  connection, concurrency: 10 });
```

Spring equivalents: `@Async` + Spring Batch + Kafka/RabbitMQ listeners. In Node, BullMQ gives retries, delays, cron repeat and rate limits out of the box.

10.6 Real-time: WebSockets and Server-Sent Events

TS

```
import { Server } from "socket.io";
const io = new Server(httpServer, { cors: { origin: "https://app.example.com" } });

io.use((socket, next) => verifyToken(socket.handshake.auth.token) ? next() : next(new Error("unauthorised")));

io.on("connection", (socket) => {
  socket.join(`user:${socket.data.userId}`);
  socket.on("chat:send", (msg) => io.to(msg.room).emit("chat:new", msg));
});

// Server-Sent Events: simple one-way push, works over plain HTTP
app.get("/events", (req, res) => {
  res.set({ "Content-Type": "text/event-stream", "Cache-Control": "no-cache", Connection: "keep-alive" });
  const t = setInterval(() => res.write(`data: ${JSON.stringify({ now: Date.now() })}\n\n`), 1000);
  req.on("close", () => clearInterval(t));
});
```

Scaling websockets across instances needs a shared adapter (Redis adapter for Socket.IO). Spring counterpart: Spring WebSocket / STOMP, **SseEmitter**, or WebFlux streaming.

10.7 Graceful shutdown and health checks

TS

```
const server = app.listen(3000);
let shuttingDown = false;

app.get("/health/ready", (_req, res) => res.status(shuttingDown ? 503 : 200).end());

async function shutdown(signal: string) {
  if (shuttingDown) return;
  shuttingDown = true;
  logger.info({ signal }, "shutting down");
  server.close(async () => { // stop accepting; finish in-flight requests
    await prisma.$disconnect();
    await redis.quit();
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10_000).unref(); // force after 10 s
}
process.on("SIGTERM", () => shutdown("SIGTERM")); // Kubernetes sends SIGTERM
process.on("SIGINT", () => shutdown("SIGINT"));
```

Spring Boot: **server.shutdown=graceful** plus **spring.lifecycle.timeout-per-shutdown-phase=20s** does the same in two lines.

10.8 Performance and memory

Problem	Symptom	Fix
Blocked event loop	Latency spikes for all requests	Move CPU work to workers; avoid sync APIs (readFileSync , bcrypt.hashSync) in handlers
Memory leak	RSS grows steadily, eventual OOM	Remove event listeners, bound caches (LRU), avoid global arrays; take heap snapshots (--inspect , Chrome DevTools)
Large JSON	Slow parse/stringify	Stream (NDJSON), paginate, use Fastify schemas
Too many DB connections	Pool exhaustion	Pool sizing, PgBouncer, reuse one client instance
Slow startup in serverless	Cold starts	Bundle with esbuild, lazy-load heavy modules

BASH

```
node --max-old-space-size=1024 dist/index.js # heap limit in MB
node --inspect dist/index.js # attach Chrome DevTools profiler
clinic doctor -- node dist/index.js # diagnose event loop / CPU / I/O
autocannon -c 100 -d 20 http://localhost:3000/api/tasks # load test
```

PRO TIP

Measure the **event loop lag** (**perf_hooks.monitorEventLoopDelay**) and export it as a metric. If p99 lag climbs, something is blocking.

10.9 Observability

TS

```
import pino from "pino";
export const logger = pino({ level: process.env.LOG_LEVEL ?? "info" }); // JSON logs, very fast

import client from "prom-client";
client.collectDefaultMetrics(); // CPU, heap, event loop lag, GC
app.get("/metrics", async (_req, res) => {
  res.set("Content-Type", client.register.contentType);
  res.end(await client.register.metrics());
});
```

Add **OpenTelemetry** ([@opentelemetry/sdk-node](https://www.npmjs.com/package/@opentelemetry/sdk-node) with auto-instrumentation) for distributed traces, equivalent to Micrometer Tracing in Spring.

10.10 Production Dockerfile

DOCKERFILE

```
FROM node:22-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:22-slim
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
COPY package.json ./
USER node
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

10.11 Advanced Node patterns worth knowing

- **Dependency injection without a framework:** pass dependencies through factory functions (`createTaskService({ repo, clock })`) - simple and highly testable.
- **AsyncLocalStorage:** per-request context (request id, user) without passing parameters; the Node equivalent of Java's `ThreadLocal`/MDC.
- **Event-driven design:** `EventEmitter` for in-process events (like Spring `ApplicationEvent`); Kafka/RabbitMQ/NATS for cross-service events.
- **Monorepos:** pnpm workspaces + Turborepo/Nx to share types (zod schemas, API contracts) between backend and frontend.
- **BFF (Backend for Frontend):** a thin Node layer that aggregates several microservices for one UI - a classic Node use case.
- **Serverless / edge:** AWS Lambda, Cloudflare Workers, Vercel Functions run JavaScript natively with near-zero cold start (Hono is popular here).

11. Deep Comparison: Spring Boot vs Node.js

CATEGORY

Decision chapter. A structured, criteria-by-criteria comparison with guidance on when each option is the optimal choice. Level: All levels.

11.1 Master comparison table

Criteria	Spring Boot (Java)	Node.js (TypeScript)	Edge
Language safety	Strong static types, mature tooling, refactoring is safe	Safe only with TypeScript strict mode; runtime is still dynamic	Spring
Time to first working API	Longer (setup, boilerplate, build)	Very short (a few lines)	Node
Productivity at scale (100k+ lines)	Structure enforced by conventions	Depends on discipline; NestJS helps	Spring
Raw CPU throughput	JIT-compiled, multi-core threads	Single thread per process	Spring
I/O concurrency	Virtual threads or WebFlux	Event loop native	Tie (Node simpler)
Memory per instance	Higher	Lower	Node
Cold start	Slower (native image fixes it)	Fast	Node
Ecosystem breadth	Enterprise: security, batch, messaging, integration	Largest package count; frontend and tooling synergy	Depends
Ecosystem quality/stability	Very stable, long-term support versions	Fast-moving; more churn and abandoned packages	Spring
Database and transactions	Best-in-class (JPA, @Transactional , Flyway)	Good (Prisma, Drizzle) but transaction handling is more manual	Spring
Real-time / WebSockets	Available, more ceremony	Natural fit (Socket.IO, ws)	Node
Full-stack code sharing	None with JS frontend	Share types and validation with React/Next.js	Node
Security framework	Spring Security: powerful, steep	Assemble yourself, more room for mistakes	Spring
Testing	Excellent slice tests, Testcontainers	Fast runners, simple mocks	Tie
Hiring pool	Large in enterprise, banks, telecom	Very large (JS developers)	Tie
Learning curve	Java + Spring + JPA + Maven	JS + async + npm + TS	Node (initially)
Deployment size	JAR ~30-80 MB, image ~200-300 MB	Image ~150-250 MB, smaller with slim/bundled	Node
Serverless suitability	Improved with native / SnapStart	Excellent	Node
Long-term maintainability	Very high	High with discipline and TypeScript	Spring

11.2 Performance in context (honest view)

- Benchmarks vary wildly with workload. For **simple JSON-over-HTTP with a database**, the database and network dominate; both stacks are usually "fast enough" and the framework choice is rarely the bottleneck.
- CPU-bound** endpoints (encryption loops, PDF/image generation, heavy calculations, big in-memory aggregations): JVM wins because it uses all cores in one process and has a mature JIT.

- **Idle-heavy / connection-heavy** services (chat, notifications, gateways): Node uses less memory per connection; Java with virtual threads is now competitive.
- **Latency stability:** Node p99 depends on avoiding event-loop blocking; JVM p99 depends on GC tuning (ZGC/G1 make this much easier today).
- **Start-up:** matters for serverless, autoscaling and developer feedback loops; Node is usually faster, Spring native images close the gap.

NOTE

Rule of thumb: optimise the **architecture** (caching, indexes, query count, batching) first. Switching languages seldom fixes a slow endpoint whose real problem is 40 SQL queries.

11.3 Cost of running (typical trend, not exact numbers)

Factor	Spring Boot	Node.js
Minimum RAM per container	~256-512 MB comfortable	~128-256 MB comfortable
CPU during warm-up	High (JIT warm-up)	Low
Instances needed for I/O-heavy load	Fewer with virtual threads, more with old thread model	Few (event loop)
Instances needed for CPU-heavy load	Fewer (multi-core)	More (one core per process)
Developer cost	Higher verbosity, more code	Less code, but more decisions and glue

11.4 Code volume: the same feature

Fetch one task, return 404 if missing:

JAVA

```
// Spring Boot: ~12 lines across classes, explicit and typed
@GetMapping("/{id}")
TaskResponse get(@PathVariable Long id) {
    return repo.findById(id).map(mapper::toResponse)
        .orElseThrow(() -> new NotFoundException("Task " + id));
}
```

TS

```
// Node: ~5 lines, less ceremony (types provided by Prisma)
app.get("/tasks/:id", async (req, res) => {
    const t = await prisma.task.findUnique({ where: { id: +req.params.id } });
    return t ? res.json(t) : res.status(404).json({ title: "Not found" });
});
```

11.5 When to choose which - decision matrix

Scenario	Best fit	Reason
Banking, insurance, ERP, large enterprise domain	Spring Boot	Transactions, security, long-term support, big-team structure
Startup MVP with a React/Next.js frontend	Node.js + TypeScript	One language, shared types, fastest iteration
Real-time chat, notifications, live dashboards	Node.js	Event-driven model, Socket.IO ecosystem
Batch processing, scheduled ETL, heavy computation	Spring Boot	Spring Batch, multithreading, JVM performance

Scenario	Best fit	Reason
API gateway / BFF aggregating many services	Node.js	Light, async, quick to change
Microservices at large scale with many teams	Both	Pick per service; agree on contracts (OpenAPI, gRPC)
Serverless functions	Node.js (or Spring native)	Cold start speed
Team of Java developers migrating to web	Spring Boot	Reuse skills
Team of frontend developers going full-stack	Node.js / NestJS	Reuse skills
Event streaming (Kafka-heavy)	Spring Boot	Spring Kafka maturity; Kafka Streams is JVM-native
AI/LLM-integrated apps, streaming responses	Node.js (also Spring AI)	Streaming and JS SDKs, quick glue

11.6 Hybrid architecture (common in real companies)

TEXT

```

React / Next.js UI
|
Node.js BFF (Next.js route handlers or NestJS)  <- shapes data for the UI, auth cookies, SSR
|
+-----+-----+
|               |
Spring Boot services    Spring Boot services    <- core domain, payments, orders, batch
(Orders, Billing)        (Inventory)
|
PostgreSQL / Kafka / Redis

```

12. Full Example: Task API in Both Stacks

CATEGORY

Worked project. The same feature set - create, read, list, complete and delete tasks with validation and errors - implemented side by side. Use it as a template. Level: Intermediate.

12.1 API contract

Method	Path	Body	Success	Errors
GET	/api/tasks?page=0&size=20&status=OPEN	-	200 page of tasks	400
GET	/api/tasks/{id}	-	200 task	404
POST	/api/tasks	{title, priority}	201 + Location	400
PATCH	/api/tasks/{id}/complete	-	200 task	404
DELETE	/api/tasks/{id}	-	204	404

12.2 Spring Boot version (complete)

JAVA

```
// Task.java
@Entity @Table(name = "tasks")
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    @Column(nullable = false, length = 100) private String title;
    private int priority;
    @Enumerated(EnumType.STRING) private Status status = Status.OPEN;
    public enum Status { OPEN, DONE }
    protected Task() {}
    public Task(String title, int priority) { this.title = title; this.priority = priority; }
    // getters + setStatus(...)
}

// TaskRepository.java
public interface TaskRepository extends JpaRepository<Task, Long> {
    Page<Task> findByStatus(Task.Status status, Pageable pageable);
}

// DTOs
public record CreateTask(@NotBlank @Size(max = 100) String title, @Min(1) @Max(5) int priority) {}
public record TaskResponse(Long id, String title, int priority, String status) {
    static TaskResponse of(Task t) {
        return new TaskResponse(t.getId(), t.getTitle(), t.getPriority(), t.getStatus().name());
    }
}

// TaskService.java
@Service @Transactional(readOnly = true)
public class TaskService {
    private final TaskRepository repo;
    public TaskService(TaskRepository repo) { this.repo = repo; }
    public Page<TaskResponse> list(Task.Status s, Pageable p) {
        Page<Task> page = (s == null) ? repo.findAll(p) : repo.findByStatus(s, p);
        return page.map(TaskResponse::of);
    }
    public TaskResponse get(Long id) { return TaskResponse.of(find(id)); }

    @Transactional public TaskResponse create(CreateTask dto) {
        return TaskResponse.of(repo.save(new Task(dto.title(), dto.priority())));
    }
}
```

```

    }
    @Transactional public TaskResponse complete(Long id) {
        Task t = find(id); t.setStatus(Task.Status.DONE); return TaskResponse.of(t);
    }
    @Transactional public void delete(Long id) { repo.delete(find(id)); }

    private Task find(Long id) {
        return repo.findById(id).orElseThrow(() -> new NotFoundException("Task " + id + " not found"));
    }
}

// TaskController.java
@RestController @RequestMapping("/api/tasks")
public class TaskController {
    private final TaskService svc;
    public TaskController(TaskService svc) { this.svc = svc; }
    @GetMapping
    public Page<TaskResponse> list(@RequestParam(required = false) Task.Status status,
                                   @PageableDefault(size = 20) Pageable p) {
        return svc.list(status, p);
    }
    @GetMapping("/{id}") public TaskResponse get(@PathVariable Long id) { return svc.get(id); }
    @PostMapping public ResponseEntity<TaskResponse> create(@Valid @RequestBody CreateTask dto) {
        TaskResponse r = svc.create(dto);
        return ResponseEntity.created(URI.create("/api/tasks/" + r.id())).body(r);
    }
    @PatchMapping("/{id}/complete")
    public TaskResponse complete(@PathVariable Long id) { return svc.complete(id); }
    @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { svc.delete(id); }
}

```

12.3 Node.js version (Express + Prisma + Zod, complete)

TS

```

// schema.ts
import { z } from "zod";
export const CreateTask = z.object({
    title: z.string().trim().min(1).max(100),
    priority: z.number().int().min(1).max(5),
});
export const ListQuery = z.object({
    page: z.coerce.number().int().min(0).default(0),
    size: z.coerce.number().int().min(1).max(100).default(20),
    status: z.enum(["OPEN", "DONE"]).optional(),
});

// service.ts
import { prisma } from "./db";
export class NotFoundError extends Error { status = 404; }
export const taskService = {
    async list({ page, size, status }: z.infer<typeof ListQuery>) {
        const where = status ? { status } : {};
        const [content, total] = await prisma.$transaction([
            prisma.task.findMany({ where, skip: page * size, take: size, orderBy: { id: "asc" } }),
            prisma.task.count({ where }),
        ]);
        return { content, totalElements: total, totalPages: Math.ceil(total / size), page };
    },
    async get(id: number) {
        const t = await prisma.task.findUnique({ where: { id } });
        if (!t) throw new NotFoundError(`Task ${id} not found`);
        return t;
    },
    create: (data: z.infer<typeof CreateTask>) => prisma.task.create({ data }),
    async complete(id: number) {
        await this.get(id);
        return prisma.task.update({ where: { id }, data: { status: "DONE" } });
    },
    async remove(id: number) { await this.get(id); await prisma.task.delete({ where: { id } }); },
};

// routes.ts
import { Router } from "express";

```

```

export const tasks = Router();
tasks.get("/", async (req, res) => res.json(await taskService.list(ListQuery.parse(req.query))));
tasks.get("/:id", async (req, res) => res.json(await taskService.get(Number(req.params.id))));
tasks.post("/", async (req, res) => {
  const t = await taskService.create(CreateTask.parse(req.body));
  res.status(201).location(`/api/tasks/${t.id}`).json(t);
});
tasks.patch("/:id/complete", async (req, res) =>
  res.json(await taskService.complete(Number(req.params.id))));
tasks.delete("/:id", async (req, res) => {
  await taskService.remove(Number(req.params.id));
  res.status(204).end();
});

// app.ts (Express 5: async errors reach the handler automatically)
const app = express();
app.use(express.json());
app.use("/api/tasks", tasks);
app.use((err: any, _req: Request, res: Response, _n: NextFunction) => {
  if (err instanceof z.ZodError)
    return res.status(400).json({ title: "Validation failed", errors: err.flatten() });
  res.status(err.status ?? 500).json({ title: err.status ? err.message : "Internal Server Error" });
});
export { app };

```

12.4 Observations from the comparison

- The Node version is roughly **40% shorter** but leaves architecture decisions (folder layout, DI, error classes) to you.
- Spring gives you pagination (**Pageable**), 400-validation, JSON mapping and the transaction boundary for free; in Node you compose each piece (Zod, Prisma **\$transaction**).
- Both follow the same layering: **Controller/route -> Service -> Repository/ORM**. Skills transfer directly.
- Spring's **Task** entity can leak persistence concerns; Prisma models are plain objects generated from the schema.

13. Interview Q&A, Cheat Sheet and Roadmap

CATEGORY

Revision chapter. The questions most commonly asked in interviews, mistakes to avoid, quick-reference cards and a structured learning path. Level: All levels.

13.1 Spring Boot interview questions

Question	Short answer
What does <code>@SpringBootApplication</code> do?	Combines <code>@Configuration</code> , <code>@EnableAutoConfiguration</code> , <code>@ComponentScan</code>
How does auto-configuration work?	Conditional config classes loaded from <code>AutoConfiguration.imports</code> , activated by classpath and beans (<code>@ConditionalOnClass</code> , <code>@ConditionalOnMissingBean</code>)
<code>@Component</code> vs <code>@Bean</code> ?	<code>@Component</code> marks your class for scanning; <code>@Bean</code> defines a bean in a <code>@Configuration</code> method (used for third-party classes)
Constructor vs field injection?	Constructor: immutable, testable, explicit. Field: hidden dependencies, hard to test
What is a bean scope?	Lifecycle of bean instances: singleton (default), prototype, request, session
<code>@Controller</code> vs <code>@RestController</code> ?	<code>@RestController</code> = <code>@Controller</code> + <code>@ResponseBody</code>
Lazy vs eager fetching?	LAZY loads relations on access (default for collections); EAGER loads immediately. Prefer LAZY + fetch joins
What is the N+1 problem?	1 query for the list + N queries for relations. Solve with <code>join fetch</code> , entity graphs or batch fetching
Why does <code>@Transactional</code> not work sometimes?	Self-invocation bypasses the proxy; private methods; checked exceptions do not roll back by default
<code>save()</code> vs <code>saveAndFlush()</code> ?	<code>saveAndFlush</code> forces the SQL to the DB immediately; <code>save</code> waits for flush/commit
How to handle exceptions globally?	<code>@RestControllerAdvice</code> with <code>@ExceptionHandler</code> methods (return <code>ProblemDetail</code>)
What are profiles?	Named configuration sets activated per environment
What is Actuator?	Production endpoints for health, metrics, env, and more
How to secure endpoints?	Spring Security filter chain, <code>authorizeHttpRequests</code> , <code>@PreAuthorize</code>
What are virtual threads?	Lightweight JVM-managed threads that make blocking I/O scalable (Java 21)

13.2 Node.js interview questions

Question	Short answer
Is Node.js single-threaded?	JS code runs on one thread; I/O uses libuv's thread pool and OS async; you can add worker threads
Explain the event loop	Phases (timers, poll, check...) processing callbacks; microtasks (promises, nextTick) run between them

Question	Short answer
<code>process.nextTick</code> vs <code>setImmediate</code> ?	<code>nextTick</code> runs before the loop continues (microtask-like); <code>setImmediate</code> runs in the check phase
Callbacks vs Promises vs <code>async/await</code> ?	Same async concept with progressively cleaner syntax; <code>async/await</code> is sugar over promises
What is middleware?	A function (<code>req, res, next</code>) in a pipeline that can modify the request/response or stop the flow
How to handle errors in async code?	<code>try/catch</code> with <code>await</code> ; error middleware; process-level handlers for last resort
How to scale Node?	Multiple processes/containers behind a load balancer; cluster; stateless design; worker threads for CPU
What are streams?	Incremental processing of data with backpressure (Readable, Writable, Transform)
CommonJS vs ESM?	<code>require/module.exports</code> vs <code>import/export</code> ; ESM is async, standard, supports top-level <code>await</code>
How to prevent blocking the loop?	Avoid sync APIs and heavy loops; offload to workers or separate services
<code>==</code> vs <code>===</code> ?	<code>===</code> compares without type coercion; always prefer it
What is a memory leak in Node?	Objects kept reachable (listeners, caches, globals) so GC cannot free them
What is <code>package-lock.json</code> ?	Pins exact dependency versions for reproducible installs (<code>npm ci</code>)

13.3 Common mistakes

Stack	Mistake	Better approach
Spring	Returning entities directly from controllers	Return DTOs
Spring	<code>ddl-auto=update</code> in production	Flyway/Liquibase migrations
Spring	Field injection everywhere	Constructor injection
Spring	Huge <code>@Transactional</code> methods calling remote APIs	Keep transactions short; call remote services outside
Spring	EAGER fetching on collections	LAZY + targeted fetch joins
Spring	Catching <code>Exception</code> and swallowing it	Handle specifically or rethrow; log with context
Node	Forgetting <code>await</code> (floating promise)	Enable <code>@typescript-eslint/no-floating-promises</code>
Node	Synchronous crypto/file APIs in handlers	Async versions or workers
Node	Trusting <code>req.body</code>	Validate with Zod/Joi
Node	New <code>PrismaClient()</code> per request	One shared instance
Node	Storing state in memory across requests	Redis/DB (stateless servers)
Node	Using <code>any</code> everywhere in TypeScript	<code>strict: true</code> , <code>unknown</code> + narrowing

13.4 Command cheat sheet

Task	Spring Boot (Maven)	Node.js (npm)
Create project	<code>start.spring.io</code>	<code>npm init -y</code>
Run in dev	<code>./mvnw spring-boot:run</code>	<code>npm run dev</code>
Run tests	<code>./mvnw test</code>	<code>npm test</code>
Build	<code>./mvnw package</code>	<code>npm run build</code>

Task	Spring Boot (Maven)	Node.js (npm)
Add dependency	Edit <code>pom.xml</code>	<code>npm install pkg</code>
Start prod	<code>java -jar target/app.jar</code>	<code>node dist/index.js</code>
Container image	<code>./mvnw spring-boot:build-image</code>	<code>docker build .</code>
Dependency tree	<code>./mvnw dependency:tree</code>	<code>npm ls</code>
Vulnerability check	OWASP dependency-check	<code>npm audit</code>

13.5 Learning roadmap (basic to advanced)

Stage	Spring Boot	Node.js	Milestone project
1. Foundations	Java syntax, OOP, collections, streams, records	JS/TS syntax, async/await, modules	CLI tool
2. Web basics	Controllers, DI, YAML config	Express routes, middleware, env config	Hello + CRUD in memory
3. Persistence	JPA, relationships, queries, Flyway	Prisma/Drizzle, migrations, transactions	Task API with PostgreSQL
4. Quality	Validation, exception handling, JUnit, MockMvc	Zod, error middleware, Vitest, Supertest	Tested API, 80% coverage
5. Security	Spring Security, JWT, OAuth2	jose/passport, Helmet, rate limit	Login + roles
6. Production	Actuator, Docker, profiles, caching	pino, metrics, Docker, graceful shutdown	Deployed API with CI
7. Advanced	Kafka, WebFlux, virtual threads, native	Streams, workers, BullMQ, WebSockets	Event-driven service
8. Architecture	Modulith, microservices, saga/outbox	Monorepo, BFF, serverless	Multi-service system

PRO TIP

Learn **one** stack deeply first, then learn the other by mapping concepts (this guide's comparison tables). The ideas - layering, DI, validation, transactions, idempotency, observability - are the same everywhere.